

Python Coding Cheat Sheet

SYNTAX & IDIOMS

```
def f(a, *args, k=1, **kw): ...
def f(x, acc=None): ... # never acc=[]
sorted(a, key=lambda p: (-p[1], p[0]))
a.sort(reverse=True) # in place, stable, None
x = a if cond else b
for i, a_i in enumerate(a, 1):
for a_i, b_i in zip(a, b): # stops @ shorter
a, b = b, a # swap
first, *mid, last = seq # star unpack
if (n := len(s)) > 3: # walrus
for ...: ... else: ... # else = no break
a[i:j:k] a[:: -1] a[-k:] # never IndexError
[x*x for x in a if x % 2]
{k: v for k, v in pairs}
{x for x in a}
[[0]*C for _ in range(R)] # NOT [[0]*C]*R
sum(x > 0 for x in a) # count matches
any(gen); all(gen); max(a, key=f, default=0)
float('inf'); -math.inf
q, r = divmod(a, b)
-7 // 2 == -4; int(-7 / 2) == -3
-7 % 3 == 2 # sign of divisor
ord('a') == 97; chr(97) == 'a'
x & 1; x >> 1; x ^ y; 1 << k
x & (x-1) # drop low bit
x & -x # low bit
x.bit_length(); bin(5) == '0b101'
int('101', 2) == 5; int('ff', 16) == 255
```

STRINGS (IMMUTABLE)

```
s.split(); s.split(',', 1); ' '.join(parts)
s.strip(); s.lstrip('0'); s.rstrip('\n')
s.find(t) # -1 if absent
s.index(t) # ValueError if absent
s.startswith(('a', 'b')); s.endswith(x)
s.isalnum(); s.isalpha(); s.isdigit(); s.islower()
s.replace(a, b); s.count(t); s.zfill(5)
''.join(sorted(s)) # anagram key
out.append(ch); ''.join(out) # build via list
f'{x:.2f} {n:05d} {n:.,} {n:b} {s:>8} {x!r}'
```

LIST / DICT / SET

```
a.append(x); a.pop() # O(1)
a.pop(0); a.insert(0, x); x in a # O(n)
a.extend(b); a.index(x); a.remove(x)
copy.deepcopy(a)
list(zip(*m)) # transpose
[list(r) for r in zip(*m[::-1])] # rotate cw
d.get(k, 0); d.pop(k, None); k in d
d.setdefault(k, []).append(v)
for k, v in d.items(): ... # insertion order
max(d, key=d.get) # key of max value
sorted(d.items(), key=lambda kv: -kv[1])
d1 | d2 # merge, right wins
a & b; a - b; a ^ b; a <= b # subset
s.add(x); s.discard(x) # no error if absent
tuple(lst); frozenset(s) # hashable keys
```

COLLECTIONS

```
deque # O(1) both ends, O(n) middle
dq = deque(it, maxlen=k) # full: drops far end
dq.append(x); dq.appendleft(x)
dq.pop(); dq.popleft(); dq[0]; dq[-1]
dq.extend(it); dq.extendleft(it); dq.rotate(k)
Counter # dict subclass, missing key -> 0
c = Counter(s); c[x] += 1
c.most_common(k) # [(elem, cnt), ...]
c.update(it); c.subtract(it); c.total()
c1 + c2; c1 - c2 # drops <= 0
c1 == c2; +c # drops <= 0
list(c.elements()); # list repr of c
defaultdict
g = defaultdict(list); g[u].append(v)
defaultdict(int); defaultdict(set)
defaultdict(lambda: math.inf)
OrderedDict # LRU cache building block
od.move_to_end(k) # mark most recent
od.popitem(last=False) # evict oldest (FIFO)
```

HEAPQ (MIN-HEAP ON A LIST)

```
heapify(lst) # O(n), in place
heappush(h, (prio, i, item)) # i breaks ties
heappop(h) h[0] # peek
heappush(h, -x) # max-heap: negate
heappushpop(h, x); heapreplace(h, x)
nlargest(k, a, key=f); nsmallest(k, a)
heapq.merge(*sorted_lists) # lazy k-way merge
```

BISECT (SORTED LIST)

```
bisect_left(a, x) # first i with a[i] >= x
bisect_right(a, x) # first i with a[i] > x
insort(a, x) # O(n) insert, keeps sorted
bisect_left(a, x, key=f) # 3.10+
```

ITERTOOLS

```
permutations(a, r); combinations(a, r)
product(a, b); product('01', repeat=n)
accumulate(a, initial=0) # prefix sums
accumulate(a, max) # running max
groupby(s) # consecutive runs; sort first!
[(k, len(list(g))) for k, g in groupby(s)]
chain(a, b); chain.from_iterable(lists)
pairwise(a) # (a0,a1),(a1,a2)
zip_longest(a, b, fillvalue=0)
islice(it, start, stop); count(1); cycle(a)
```

FUNCTOOLS

```
@cache # memoise; args must be hashable
@lru_cache(maxsize=128); f.cache_clear()
reduce(lambda acc, x: acc * x, a, 1)
sorted(a, key=cmp_to_key(cmp))
# cmp(x, y) < 0: x first
partial(f, 1, k=2)
```

URLLIB.PARSE

```
u = urlparse(
'https://u:p@H.com:8443/a/b?x=1&x=2&y=#frag')
u.scheme 'https'; u.netloc 'u:p@H.com:8443'
u.hostname 'h.com'; u.port 8443; u.path '/a/b'
u.query 'x=1&...'; u.fragment 'frag'
u.username 'u'; u.password 'p'
u._replace(query='', fragment='').geturl()
parse_qs(u.query) # {'x': ['1', '2']}
parse_qs(q, keep_blank_values=True) # keeps y
parse_qsl(u.query) # [('x','1'),('x','2')]
urlencode({'q': 'a b', 't': [1,2]}, doseq=True)
# 'q=a+b&t=1&t=2'
quote('a b/c') # 'a%20b/c'
quote('a b/c', safe='') # 'a%20b%2Fc'
quote_plus('a b&c') # 'a+b%26c'
unquote('a%20b'); unquote_plus('a+b') # 'a b'
urljoin('http://h/a', 'c') # http://h/c
urljoin('http://h/a/', 'c') # http://h/a/c
```

URLLIB.REQUEST / URLLIB.ERROR

```
with urlopen(url, timeout=5) as r: # GET
r.status; r.headers['Content-Type']
data = json.load(r) # or r.read().decode
req = Request(url, method='POST',
data=json.dumps(body).encode(), # bytes
headers={...})
with urlopen(req, timeout=5) as r:
print(r.read().decode())
from urllib.error
import HTTPError, URLError,
TimeoutError, URLError
e.code; e.reason; e.read()
```

CLASSES, GENERATORS, ERRORS

```
class Job:
def __lt__(self, o): return self.v < o.v # heap
def __eq__ + def __hash__ # custom dict key
def __repr__(self): return f'N({self.v})'
def gen(): yield x;
next(it, default) # no StopIteration
try: ...
except (KeyError, ValueError) as e: ...
else: ... # no exception
finally: ... # always
raise ValueError('msg')
with open(p) as f:
for line in f: line.rstrip('\n')
```

RE / TIME / CONCURRENCY

```
re.findall(r'\d+', s); re.split(r'[;, ]+', s)
p = re.compile(r'\d+'); m = p.match(s);
m = re.match(r'(\w+)=(\d+)', s); m.group(1)
re.sub(r'\s+', ' ', s); re.fullmatch(pat, s)
time.monotonic() # durations; time.time()=wall
datetime.now(timezone.utc).isoformat()
(t2 - t1).total_seconds()
lock = threading.Lock()
with lock: ... # guard shared state
with ThreadPoolExecutor(max_workers=8) as ex:
for r in ex.map(fetch, urls): ... # ordered
futs = [ex.submit(fetch, u) for u in urls]
for fu in as_completed(futs): fu.result()
```